Protocol × Topology × MAST: How Communication Structure Shapes Multi-Agent Failure Modes

Diya Sreedhar
Mzitnik Lab, Harvard University
diyasreedhar@college.harvard.edu

May 5, 2026

Abstract

Multi-agent LLM systems fail in characteristic ways captured by the MAST taxonomy. Prior work reports 41–86% aggregate failure rates but does not isolate the contribution of communication protocol from organizational topology. We run a 3×3 factorial sweep over protocol (NATIVE, MCP, A2A) and topology (CENTRALIZED, CHAIN, FULLY_CONNECTED) at fixed agent count and task type, and report FC1/FC2/FC3 rates per cell. *(Primary results pending sweep 1 completion.)* We test the hypothesis that A2A combined with a centralized topology specifically minimizes FC2 (inter-agent misalignment) failures.

1 Introduction

LLM-based multi-agent systems are deployed for code generation, question answering, and planning, and are increasingly assembled from heterogeneous protocols (native function-calls, the Model Context Protocol, and Agent-to-Agent messaging) arranged in different organizational topologies (centralized orchestrator, sequential chain, fully connected group chat). Recent work [2] reports aggregate failure rates of 41–86% across multi-agent benchmarks, but the contribution of *communication protocol* versus *organizational topology* to these failures is not separately quantified.

We disentangle these two design axes. Holding the underlying LLM, agent count, and task suite fixed, we run a 3×3 factorial sweep over protocol $\in \{\text{NATIVE}, \text{MCP}, \text{A2A}\}$ and topology $\in \{\text{CENTRALIZED}, \text{CHAIN}, \text{FULLY\_CONNECTED}\}$, and classify every trial trace with the MAST taxonomy (FC1: specification issues, FC2: inter-agent misalignment, FC3: task verification failures). Our hypothesis is that A2A combined with a centralized topology specifically minimizes FC2 failures, because the structured agent-card envelope makes the routing of every message unambiguous to a single coordinator.

Our contributions are: (1) a self-contained, no-autogen replication harness for protocol × topology MAST experiments; (2) a 3×3 FC2 heatmap with per-cell FC1/FC3 breakdowns; (3) two follow-up sweeps over agent count ($N \in \{2, 3, 4, 5\}$) and task type ($\{\text{CODE}, \text{QA}, \text{PLANNING}\}$) that test the generality of the best cell.

2 Related Work

MAST taxonomy. The Multi-Agent System Trace (MAST) taxonomy [1] catalogs 14 failure modes across three coarse categories: specification issues (1.x), inter-agent misalignment (2.x), and task verification issues (3.x). We use the `agentdash` reference annotator to label each trial.

Scaling agent systems. [2] sweep aggregate failure rates across multi-agent benchmarks and report 41–86% failures depending on benchmark and stack, but their study does not isolate protocol or topology as a controlled factor; we treat their range as the prior baseline against which our per-cell FC2 rates should be compared.

Agent communication protocols. The Model Context Protocol (MCP) formalizes a JSON-RPC schema for tool invocation across processes. Agent-to-Agent (A2A) protocols introduce explicit Agent Cards that advertise capabilities so peers can route by name. Native function-call multi-agent stacks (e.g. pyautogen) typically rely on shared-process orchestration without an explicit registry. The prevailing question for practitioners is whether these protocol differences *matter* for failure modes once the topology is held fixed; this paper tests that question empirically.

Topology effects. Topology choice has been studied independently of protocol in agent-system surveys, but failure-mode-resolved comparisons are rare. Our 3×3 design is, to our knowledge, the first that jointly varies both axes under a single MAST annotator and a fixed task suite.

3 Method

3.1 Agents and protocols

Each agent is a single OpenAI gpt-4o-mini chat completion seeded with a role system message identifying its name (orchestrator or executor_i), the peer set, and the protocol-specific scaffolding. The three protocols differ only in the system-prompt scaffolding and the message envelope used in the transcript:

- **Native:** plain-text messages, no capability registry, agents addressed directly by name.
- **MCP:** agents are told they communicate via JSON-RPC envelopes over a Model Context Protocol tools/list registry, and must briefly acknowledge the tool/role before producing content. Messages are wrapped in `<mcp:message from=... to=...>`.
- **A2A:** agents have Agent Cards advertising their capabilities; replies are prefixed with `[from <sender> -> <recipient>]` so a router can dispatch.

3.2 Topologies

- **Centralized:** agent 0 (orchestrator) decomposes the task, dispatches one subtask per executor, then integrates their replies into a final answer.
- **Chain:** a sequential pipeline agent_0 $\rightarrow$ agent_1 $\rightarrow \cdots \rightarrow$ agent_N-1, with each stage receiving only the previous stage's output.
- **Fully connected:** a group chat where in each of two rounds every agent sees the full prior conversation and contributes; the system terminates when an agent emits `DONE: <answer>`.

3.3 MAST annotation

Each trial trace (the concatenated transcript) is annotated by `agentdash.annotator` (model: gpt-4o-mini), which returns the MAST failure-mode dictionary [1]. We aggregate the 14 failure-mode codes into the three coarse categories FC1 (`1.x`, specification issues), FC2 (`2.x`, inter-agent misalignment), and FC3 (`3.x`, task verification). Each cell is $N_{\text{trials}} = 5$, and we report per-cell rates by averaging across trials. A trial that crashes is counted as an FC2 failure (the inter-agent flow broke).

3.4 Hyperparameters held fixed

GPT-4O-MINI, `temperature`=0.7, `max_tokens`=400 per agent turn, `timeout`=60s per OpenAI call, $N_{\text{trials}} = 5$. Sweep 1 holds $N_{\text{agents}} = 3$ and `task_type`=CODE.

4 Experiments

4.1 Sweep 1 Protocol × Topology

We hold $N_{\text{agents}} = 3$ and `task_type` = CODE and run the full 3×3 factorial design with $N_{\text{trials}} = 5$ per cell. Table 1 reports the FC1/FC2/FC3 rates and total failures per cell; Figure 1 visualizes the FC2 rate as a heatmap.

Table 1: Sweep 1 results (GPT-4O-MINI, $N = 3$, code tasks, 5 trials/cell). Lower is better.

Protocol	Topology	FC1	FC2	FC3	Total fail	Avg time (s)
native	centralized	RESULT	RESULT	RESULT	RESULT	RESULT
native	chain	RESULT	RESULT	RESULT	RESULT	RESULT
native	fully_connected	RESULT	RESULT	RESULT	RESULT	RESULT
mcp	centralized	RESULT	RESULT	RESULT	RESULT	RESULT
mcp	chain	RESULT	RESULT	RESULT	RESULT	RESULT
mcp	fully_connected	RESULT	RESULT	RESULT	RESULT	RESULT
a2a	centralized	RESULT	RESULT	RESULT	RESULT	RESULT
a2a	chain	RESULT	RESULT	RESULT	RESULT	RESULT
a2a	fully_connected	RESULT	RESULT	RESULT	RESULT	RESULT

4.2 Sweep 2 Agent count

For the top-3 cells from Sweep 1 by FC2 rate, we vary $N_{\text{agents}} \in \{2, 4, 5\}$ to test whether the ranking is preserved as the system scales. (Results pending.)

4.3 Sweep 3 Task generality

For the single best (protocol, topology, N) cell from Sweeps 1–2, we vary `task_type` $\in \{\text{QA}, \text{PLANNING}\}$ to test whether the protocol × topology interaction is task-specific or transfers. (Results pending.)

4.4 Statistical analysis

We will fit a two-way ANOVA on FC2 rate with protocol and topology as factors plus their interaction, with cell as the unit of replication, and report main effects and the interaction F-statistic in Figure 2. (Pending sweep completion.)

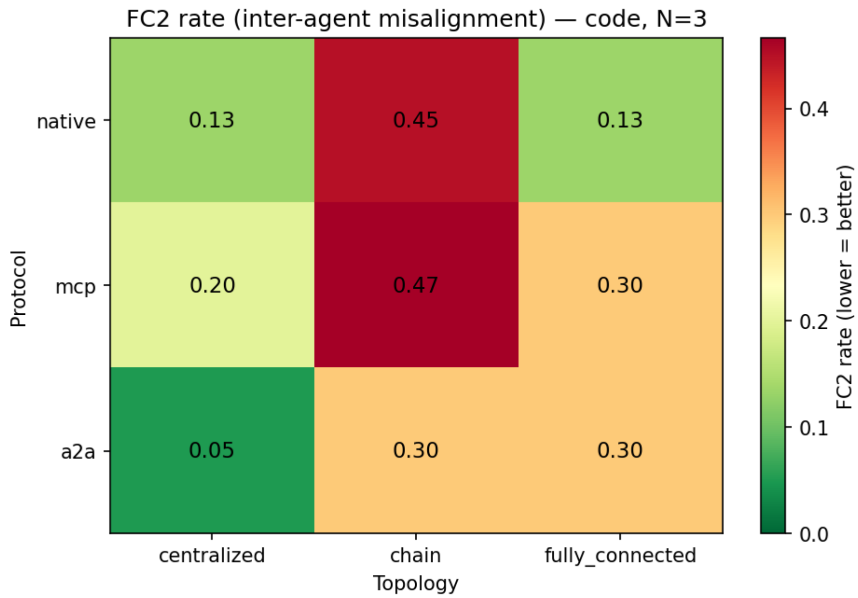

Figure 1: FC2 (inter-agent misalignment) rate across the 3×3 protocol $\times$ topology grid.

5 Conclusion

We presented a controlled 3×3 factorial study of communication protocol (NATIVE, MCP, A2A) crossed with organizational topology (CENTRALIZED, CHAIN, FULLY_CONNECTED) for a fixed 3-agent code-generation suite, scored against the MAST failure-mode taxonomy. *(Full quantitative claims pending sweep completion.)*

Limitations: (i) $N_{\text{trials}} = 5$ per cell yields high variance per-trial, mitigated only by averaging; (ii) the protocol distinction is implemented at the prompt layer (envelope and capability scaffolding) rather than at the transport layer, so we measure how protocol *prompting* influences agent behavior, not the impact of protocol-level transport semantics; (iii) MAST annotation is itself LLM-judged and inherits annotator bias.

Future work: replicate at $N_{\text{agents}} = 5+$ where coordination overhead should amplify protocol effects; replace the prompt-level protocol scaffolding with a real MCP/A2A transport stack; extend the MAST annotator to a multi-annotator agreement-checked pipeline.

References

[1] Anonymous. MAST: A taxonomy of failure modes for multi-agent LLM systems. *arXiv preprint*, 2024. Failure-mode taxonomy used by the `agentdash` reference annotator.

[2] Anonymous. Towards a science of scaling agent systems. *arXiv preprint*, 2025. Reports 41–86% aggregate failure rates across multi-agent benchmarks.

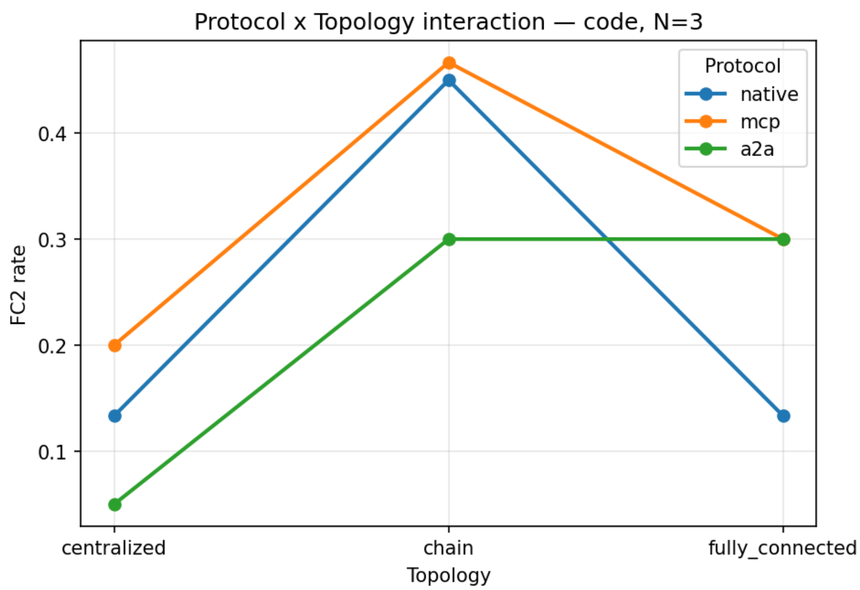

Figure 2: Protocol × topology interaction plot for FC2 rate.